

인공지능개론 · Team Project

Audio Classification Projects

직접 모으고, 직접 라벨링하기

전해곤 교수

연세대학교 첨단컴퓨팅학부



이 프로젝트에서 경험할 것

모델을 돌리는 경험

- 오디오를 spectrogram으로 바꾼다.
- ResNet-18 baseline을 학습한다.
- accuracy와 macro-F1을 비교한다.

데이터를 책임지는 경험

- 수집 기준과 label 규칙을 정한다.
- annotation 불일치를 해결한다.



연세대학교 첨단컴퓨팅학부

세 가지 프로젝트 중 선택

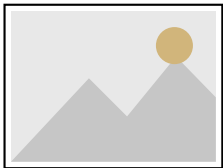


연예인 목소리 · 10개 keyword + silence · 강아지 10품종



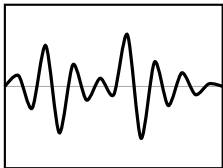
이미지는 이미지인데, 오디오는 어떻게 쓸까?

이미지



가로 × 세로 × 색상

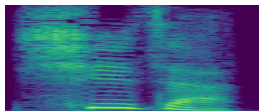
오디오



시간 × 진폭



Spectrogram

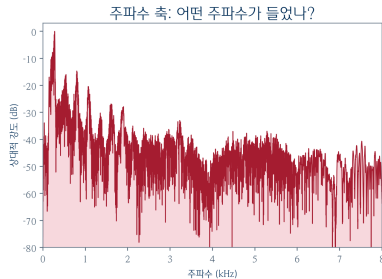
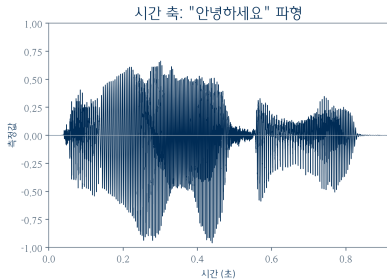


시간 × 주파수 × 강도

오디오는 Spectrogram으로 바꾸어 이미지처럼 쓴다!



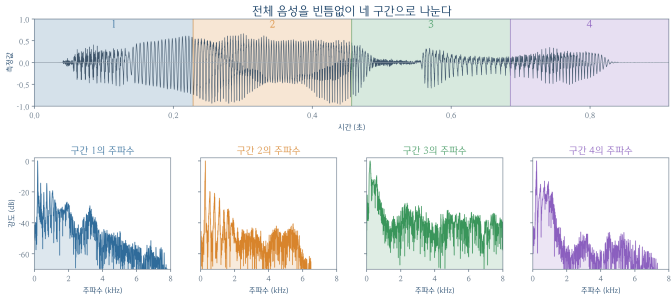
1. 푸리에 변환: 주파수별로 펼쳐 보기



푸리에 변환은 시간에 따른 신호를
"어떤 주파수가 얼마나 들었는가?"라는 관점으로 바꿔 본다.



2. 전체 음성을 짧게 잘라서 반복한다

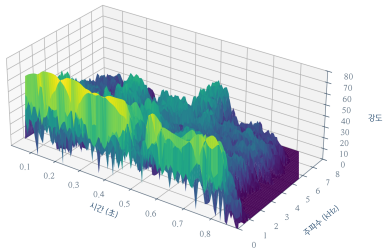


음성을 아주 짧은 구간으로 나누고,
각 구간에 푸리에 변환을 반복한다.



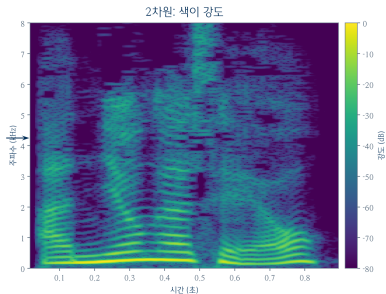
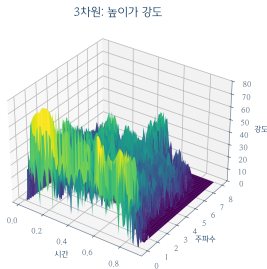
3. 구간별 결과를 시간 순서대로 이어 붙인다

구간별 결과를 시간 순서대로 이어 붙인 3차원 표현



시간 × 주파수 × 강도를 함께 보면 3차원 모양이 된다.

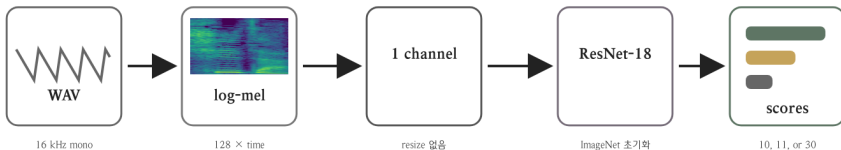
4. 위에서 내려다보면 Spectrogram!



**3차원의 높이를 색으로 바꾸면
2차원 Spectrogram으로 표현할 수 있다.**



공통 baseline 파이프라인





데이터셋의 설명서 metadata.csv

path	label	group	split	source_url / license	notes
clip_001.wav	정지	speaker_03	train	URL · 어용 범위	집음 약간
clip_002.wav	정지	speaker_05	val	직접 녹음 · 동의	정상
clip_003.wav	시작	speaker_01	test	직접 녹음 · 동의	짧음



Audio를 위해서는 ResNet-18을 1채널로 쓴다



```
model.conv1 = nn.Conv2d(1, 64, 7, 2, 3, bias=False)
model.fc = nn.Linear(512, num_classes)
```



accuracy 하나만 보면 놓치는 것

Accuracy

전체 clip 중 맞힌 비율. class 수가 불균형하면 큰 class가 점수를 지배한다.

Macro-F1

class별 F1을 같은 비중으로 평균. 작은 class의 실패도 드러난다.

Confusion matrix

어떤 class 쌍을 자주 혼동하는지 보여 준다.

test 사용 규칙

모델 · epoch · augmentation 선택은 validation으로 한다. test는 모든 결정이 끝난 뒤 마지막에 한 번만 평가한다.

최종 보고: accuracy + macro-F1 + confusion matrix + 오분류 20개 청취 분석



Project A · 연예인 30명 목소리 분류



문제

3초 음성만 듣고 30명 중 누구인지 고른다.

기술적 명칭

closed-set speaker identification

목표

얼굴 · 이름 · 자막이 아니라 목소리 패턴으로 분류한다.



Project A · 제안하는 유명인 30명 (학생들이 자유롭게 30명 선택 가능)

배우 10명



이병헌



송강호



황정민



전지현



손예진



김예수



공유



박보영



마동석



김고은

아이돌 10명



아이유



태연



윤아



차은우



전예곤



RM



뷔



카리나



장원영



제니

개그맨 10명



유재석



강호동



신동엽



이경규



박명수



김구라



양세형



장도연



박나래



조세호



Project A · 데이터 목표

권장 최소량

- 30명 × 30 usable clips = 900 clips
- 인물마다 독립 source 영상 10개 이상
- clip 길이 2~4초, 목표 3초
- interview · 방송 · 행사 등 source 다양화

수집 단위

- 한 영상에서 최대 3 clip만 사용
- 배경 음악과 다른 화자 겹침 최소화
- 원본 영상 ID를 반드시 group으로 기록
- 동일 발언 재업로드는 중복 제거



Project A · annotation 기준

포함

- 대상 인물이 혼자 말하는 구간
- 발화가 1초 이상 명확한 구간
- source와 인물 확인이 가능한 구간

제외

- 노래·성대모사·음성 변조
- 다른 사람과 심하게 겹치는 발화
- 배경 음악이 목소리보다 큰 구간
- 신원을 확인할 수 없는 재편집 영상



Project A · 모델이 목소리 대신 무엇을 외울까?

방송 환경

특정 인물의 clip이 모두 같은 프로그램이면 배경음·마이크를 외울 수 있다.

말의 내용

한 인물만 같은 유행어나 인사말을 반복하면 단어를 외울 수 있다.

데이터 양

유명한 인물의 clip만 많으면 class imbalance가 생긴다.



“오이쉬에~”

반드시 비교할 실험

random clip split과 source-video split의 성능을 모두 측정한다. 성능이 크게 떨어지면 baseline이 source를 외웠다는 증거다.



Project B · 10개 Keyword + silence



문제

짧은 음성에서 10개 명령어 또는
“아무 키워드도 없음”을 고른다.

예시

시작 · 정지 · 확인 · 취소 · 도움 ·
다음 · 이전 · 켜기 · 끄기 · 저장

핵심

새로운 화자에게도 작동하는가?



Project B · 좋은 키워드를 정하는 법

좋은 조합

- 실제 사용 장면이 분명하다.
- 길이와 음절 구조가 다양하다.
- 발음이 지나치게 비슷하지 않다.
- 모든 팀원이 자연스럽게 말할 수 있다.

일부러 어려운 조합

- 켜 / 꺼, 일 / 이, 전 / 정
- 시작 / 다시 시작
- 짧은 1음절 단어만 10개
- 외래어와 한국어가 뒤섞인 경우

쉬운 단어만 골라 점수를 높이기보다, 혼동 쌍을 포함하고 왜 틀리는지 분석한다.



Project B · 5인 팀 녹음 프로토콜

기본 수집

- $5\text{명} \times 10\text{단어} \times 20\text{회} = 1,000\text{ clips}$
- 각 환경에서 화자 없는 silence도 별도 수집
- 조용한 방/복도/야외 등 3환경
- 스마트폰 거리를 20/50/100 cm로 변화
- 정상/작게/빠르게 말하기를 섞음

split

- train: 3명
- validation: 1명
- test: 1명

keyword는 화자별로, silence는 room session별로 split을 분리한다.



Project B · baseline과 현실 사이

필수 baseline: 11 classes

10 keywords + silence. 아무 말이 없을 때 keyword를 강제로 출력하지 않는다.

선택 확장: 12 classes

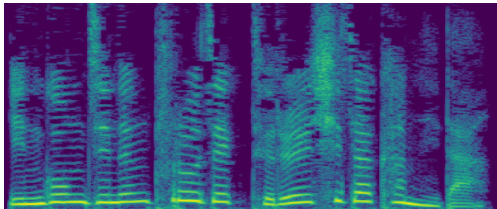
필수 11 classes에 unknown을 더한다. 등록되지 않은 말과 무음도 서로 다른 입력이다.

unknown 수집 예시

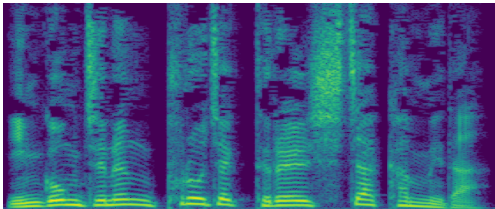
팀원이 자유롭게 말한 다른 단어, 짧은 문장, 기침 · 웃음. 필수 silence는 방별 배경음만 녹음한다.



Project B · TTS 합성 음성은 데이터 개수에 포함하지 않는다



여성 TTS: “인공지능 프로젝트를 시작합니다”



남성 TTS: 같은 문장

TTS만으로 학습하면 실제 학생 녹음에 일반화하기 어렵다.



Project C · 강아지 10개 품종 울음소리 분류



소리만 듣고 품종을 맞힐 수 있을까?



Project C · 제안하는 10개 품종

1. 치와와
2. 골든 리트리버
3. 말티즈
4. 시바견
5. 시베리안 허스키
6. 저먼 셰퍼드
7. 비글
8. 푸들
9. 웰시 코기
10. 프렌치 불도그

class 확정 전 pilot

품종당 source 영상 10개 이상과 usable bark 50개 이상을 확보할 수 있는지 먼저 확인한다.
부족하면 품종 목록을 조정한다.



Project C · 공개 데이터와 직접 annotation을 결합

Barkopedia / DogSpeak

- 156마리, 5개 품종
- Chihuahua, German Shepherd, Husky, Pitbull, Shiba Inu
- 공개 train data로 pipeline 점검 가능

팀의 확장 데이터

- 나머지 품종을 직접 수집·검수
- 영상에서 품종을 시각적으로 확인
- dog ID가 불명확하면 source video를 group으로 사용



Project C · annotation 기준

포함

- 화면에서 품종과 발성 개체 확인
- bark/howl/whine 유형을 notes에 기록
- 1~4초의 명확한 vocalization
- source URL과 clip 시간을 기록

제외

- 혼종 또는 품종 확인 불가
- 여러 강아지가 동시에 짖는 구간
- 사람 말·음악이 더 큰 구간
- 동일 영상 재업로드 또는 합성 음원

품종 label과 소리 구간 label은 서로 다른 annotation 작업이다.



Project C · 모델이 품종 대신 무엇을 외울까?

개체 identity

한 품종에 강아지 한 마리만
있으면 그 개체를 외운다.

촬영 환경

품종마다 다른 채널·마이크·
배경이면 source를 외운다.

발성 맥락

어떤 품종은 howl만, 다른
품종은 bark만 있으면 발성
유형을 외운다.

가장 강한 평가

train에 없는 개체를 test에 둔다. dog ID가 없으면 최소한 원본 source 영상 전체를 한 split에만
둔다.



저작권·동의·오용 방지

연예인 음성·웹 오디오

- 공개·허용된 source와 수업 목적 범위만 사용
- URL·제공자·license·접근일 기록
- 재배포가 불가하면 원본 파일을 공개하지 않음
- 음성 복제·사칭·신원 추적에 사용하지 않음

학생 녹음·동물 영상

- 참여 동의와 철회·삭제 절차 마련
- 실명 대신 익명 speaker ID 사용
- 동물을 자극해 새 음원을 만들지 않음
- 불확실한 품종·개체는 제외



Appendix · Notebook 사용법

더미 데이터 실행부터 결과 확인까지



연세대학교 첨단컴퓨팅학부

제공 파일

audio_classification_resnet18_dummy_dataset.ipynb

더미 데이터로 실제 실행할 notebook

dummy_datasets/

세 과제의 sanity check용 metadata와 WAV

metadata_template.csv

실제 데이터를 만들 때 사용할 annotation 양식

처음에는 제공한 더미 데이터와 설정을 바꾸지 않고 끝까지 실행한다.



프로젝트 폴더 구조

```
audio_classification_projects/  
notebooks/
```

```
audio_classification_resnet18_  
dummy_dataset.ipynb  
dummy_datasets/  
celebrity_voice/  
keyword_spotting/  
dog_breed/  
best_resnet18_audio.pt  
error_analysis.csv
```

- 폴더 이름은 대소문자까지 정확히 맞춘다.
- metadata의 path는 audio/부터 적는다.
- 공백과 한글 파일명은 가능하지만 단순한 영문 이름을 권장한다.
- MP3보다 16 kHz mono WAV가 오류가 적다.



metadata.csv 초안 생성

```
rows = []
for wav in sorted((PROJECT_ROOT / "audio")
                  .glob("*/*.wav")):
    rows.append({
        "path": wav.relative_to(PROJECT_ROOT).as_posix(),
        "label": wav.parent.name,
        "group": wav.stem,
        "split": "",
    })

pd.DataFrame(rows).to_csv(
    PROJECT_ROOT / "metadata_draft.csv",
    index=False, encoding="utf-8-sig")
```

1. 먼저 class별 폴더로 WAV를 정리한다.
2. 코드를 실행해 path와 label 초안을 만든다.
3. group을 실제 speaker, source video, dog ID로 고친다.
4. 출처 · license · annotator를 채우고 검수한다.

파일명은 group의 정답이 아니다. 생성 뒤 실제 source ID를 확인한다.



group과 split의 차이

group

같은 사람 · 원본 영상 · 강아지에서 나온 clip을 묶는 ID

split

데이터를 사용할 단계: train, val, test

path	label	group	split
start_01.wav	시작	speaker_01	train
stop_01.wav	정지	speaker_01	train
help_01.wav	도와줘	speaker_01	train
start_07.wav	시작	speaker_07	test

같은 group의 모든 파일은 반드시 하나의 split에만 들어간다.



1 · 과제와 경로 설정

Py audio_classification_resnet18_dummy_dataset.ipynb

Dummy dataset notebook

1. 과제와 설정 선택

TASK와 경로만 바꾸고 나머지는 baseline 값으로 시작합니다.

```
[ ] TASK_PRESETS = {  
    "celebrity_voice": {"num_classes": 30, "seconds": 3.0, "group_meaning": "source_video_id"},  
    "keyword_spotting": {"num_classes": 11, "seconds": 1.5, "group_meaning": "speaker_id or room_session_id"},  
    "dog_breed": {"num_classes": 10, "seconds": 3.0, "group_meaning": "dog_id or source_video_id"},  
}  
  
@dataclass  
class Config:  
    task: str = TASK  
    data_root: str = str(PROJECT_ROOT)  
    metadata_csv: str = str(PROJECT_ROOT / "metadata.csv")  
    sample_rate: int = 16_000  
    n_fft: int = 512  
    win_length: int = 512  
    hop_length: int = 160  
    n_mels: int = 128  
    batch_size: int = 32  
    epochs: int = 1  
    learning_rate: float = 3e-4  
    num_workers: int = 0  
    use_pretrained: bool = False  
    use_augmentation: bool = True  
  
CFG = Config()  
PRESET = TASK_PRESETS[CFG.task]
```

처음에는 epoch, learning rate, augmentation을 바꾸지 않고 baseline을 끝까지 실행한다.



TASK 선택

```
TASK = ``celebrity_voice``
```

```
TASK = ``keyword_spotting``
```

```
TASK = ``dog_breed``
```

자동으로 달라지는 것

class 수와 clip 길이, group의 의미

직접 확인할 것

metadata의 실제 class 수가 preset과 같은지

연예인은 30 classes, keyword는 11 classes, dog breed는 10 classes여야 한다.



더미 데이터 실행

제공한 sanity dataset

연예인 30명 × TTS 3개, keyword 10개 ×
TTS 3개 + silence 3개, dog breed 10개 ×
합성 bark 3개

확인할 것

metadata 읽기, WAV loading, spectrogram
shape, ResNet forward/backward, 결과 파일
저장

```
TASK = ``keyword_spotting``  
PROJECT_ROOT = PROJECT_DIR / ``dummy_datasets`` / TASK  
CFG = Config(task=TASK, epochs=1)  
  
# TASK = ``celebrity_voice`` 또는 ``dog_breed``
```

합성 데이터의 accuracy는 의미가 없다. 코드가 처음부터 끝까지 실행되는지만 확인한다.



더미 데이터셋 실행 검증

```
        'y': ys,
        'pred': preds,
        'paths': paths,
    }

best_state, best_f1 = None, -1.0
history = []
for epoch in range(1, CFG.epochs + 1):
    model.train()
    running = 0.0
    for x, y, _ in loaders["train"]:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad(set_to_none=True)
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
        running += loss.item() * len(y)
    val = evaluate(loaders["val"])
    train_loss = running / len(datasets["train"])
    history.append({"epoch": epoch, "train_loss": train_loss, **{f"val_{k}": v for k, v in val.items() if k in ["loss", "accuracy", "macro_f1"]}})
    print(f"{epoch:02d} train_loss={train_loss:.4f} val_acc={val['accuracy']:.3f} val_macro_f1={val['macro_f1']:.3f}")
    if val["macro_f1"] > best_f1:
        best_f1 = val["macro_f1"]
        best_state = copy.deepcopy(model.state_dict())
    scheduler.step()

model.load_state_dict(best_state)
torch.save({"model": model.state_dict(), "labels": LABELS, "config": CFG.__dict__, "best_resnet18_audio.pt"})

01 train_loss=2.4233 val_acc=0.091 val_macro_f1=0.036
```

ResNet-18 학습과 validation 출력까지 실제로 확인했다.



2 · metadata와 group split 확인

Py

audio_classification_resnet18_dummy_dataset.ipynb

Dummy dataset notebook

2. metadata와 group split

같은 group이 train/validation/test에 겹치지 않는지 코드가 검사합니다.

```
[ ] def make_group_splits(df: pd.DataFrame) -> pd.DataFrame:
    """Create 60/20/20-ish splits without sharing a group across splits."""
    df = df.copy()
    required = {"path", "label", "group"}
    missing = required - set(df.columns)
    if missing:
        raise ValueError(f"metadata.csv에 필요한 열이 없습니다: {sorted(missing)}")
    if df[["path", "label", "group"]].isna().any().any():
        raise ValueError("path, label, group에는 빈 값이 없어야 합니다.")
    if "split" in df and df["split"].notna().any():
        if not df["split"].notna().all() or set(df["split"].unique()) != {"train", "val", "test"}:
            raise ValueError("split을 직접 지정할 때는 모든 행을 train/val/test 중 하나로 채우세요.")
        return df

    per_label_groups = df.groupby("label")["group"].nunique()
    if per_label_groups.min() < 5:
        raise ValueError(
            "자동 split에는 label마다 서로 다른 group이 최소 5개 필요합니다. "
            "직접 split을 지정하거나 데이터를 더 모으세요.\n" + str(per_label_groups)
        )
    outer = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEED)
    train_val_idx, test_idx = next(outer.split(df, df.label, groups=df.group))
    train_val = df.iloc[train_val_idx]
    inner = StratifiedGroupKFold(n_splits=4, shuffle=True, random_state=SEED + 1)
    train_rel, val_rel = next(inner.split(train_val, train_val.label, groups=train_val.group))
```

출력된 표에서 모든 label에 train, validation, test clip이 있는지 확인한다.



split 검사 보호 장치

“label마다 group이 5개 필요”

한 인물·화자·영상에서 clip만 많이 자른 것으로는 새로운 source 성능을 평가할 수 없다. group을 더 모으거나 split을 직접 지정한다.

“class 수가 다름”

TASK preset의 10/30과 실제 label 수가 다르다. 오타와 빈 class를 확인한다.

“group leakage”

같은 speaker, source video, dog ID가 여러 split에 들어갔다. metadata의 split 값을 수정한다.

“파일을 찾을 수 없음”

PROJECT_ROOT와 metadata의 상대 path를 이어 붙였을 때 실제 WAV 위치가 되는지 확인한다.



3 · WAV → log-mel spectrogram

Py audio_classification_resnet18_dummy_dataset.ipynb

Dummy dataset notebook

3. WAV에서 log-mel 만들기

오디오는 학습할 때 1-channel spectrogram tensor로 변환됩니다.

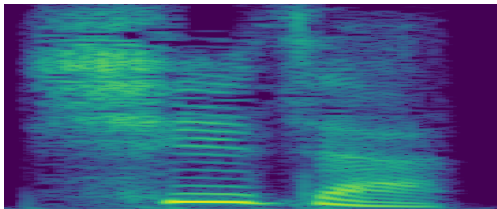
```
[ ] class AudioDataset(Dataset):
    self.mel = torchaudio.transforms.MelSpectrogram(
    self.to_db = torchaudio.transforms.AmplitudeToDB(stype="power", top_db=80)
    audio, sr = sf.read(path, dtype="float32", always_2d=True)
    wav = torchaudio.functional.resample(wav, sr, CFG.sample_rate)
    spec = self.to_db(self.mel(wav))
    spec = ((spec + 80.0) / 80.0).clamp(0, 1)
    spec = torchaudio.transforms.FrequencyMasking(12)(spec)
    spec = torchaudio.transforms.TimeMasking(18)(spec)
    spec = (spec - 0.5) / 0.25
    return spec, label_to_idx[row.label], str(row.path)
```

resize 없이 [1, 128, T]를 사용합니다. T는 프레임 고정 clip 길이에 따라 정해집니다.

Dataset은 resize 없이 [1, 128, T] log-mel을 반환한다. T는 고정된 clip 길이에 따라 정해진다.



입력 batch 확인



정상적인 speech spectrogram 예시

- 그림이 모두 검거나 모두 같은 모양은 아닌가?
- label index가 metadata label과 맞는가?
- keyword clip이 앞뒤 무음만으로 채워지지 않았는가?
- waveform peak가 0이거나 clipping되지 않았는가?

학습 전에 입력과 label을 직접 확인하면 많은 오류를 GPU를 쓰기 전에 찾을 수 있다.



4 · 1-channel ResNet-18



audio_classification_resnet18_dummy_dataset.ipynb

Dummy dataset notebook

4. ResNet-18을 1채널로 변경

ImageNet의 첫 층 weight를 채널 평균하여 spectrogram 입력에 사용합니다.

```
[ ] def build_model(num_classes: int) -> nn.Module:
    weights = ResNet18_Weights.DEFAULT if CFG.use_pretrained else None
    model = resnet18(weights=weights)
    old_weight = model.conv1.weight.detach().mean(dim=1, keepdim=True)
    model.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
    model.conv1.weight.data.copy_(old_weight)
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model
print(model.conv1)
print(model.fc)
```

첫 층과 마지막 분류 층만 바꾸고 나머지는 같은 ResNet-18입니다.



5 · 학습과 validation

Py audio_classification_resnet18_dummy_dataset.ipynb

Dummy dataset notebook

5. 학습과 validation

매 epoch마다 validation macro-F1을 계산해 가장 좋은 checkpoint를 남깁니다.

```
[ ] for epoch in range(1, CFG.epochs + 1):
    model.train()
    running = 0.0
    for x, y, _ in loaders["train"]:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad(set_to_none=True)
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
        running += loss.item() * len(y)
    val = evaluate(loaders["val"])
    train_loss = running / len(datasets["train"])
    history.append({"epoch": epoch, "train_loss": train_loss, **{f'val_{k}': v for k, v in val.items() if k in
["loss", "accuracy", "macro_f1"]}}})
    print(f'{epoch:02d} train_loss={train_loss:.4f} val_acc={val["accuracy"]:.3f} val_macro_f1={val["macro_f1"]:.3f}')
    if val["macro_f1"] > best_f1:
        best_f1 = val["macro_f1"]
        best_state = copy.deepcopy(model.state_dict())
        scheduler.step()

    model.load_state_dict(best_state)
    torch.save({"model": model.state_dict(), "labels": LABELS, "config": CFG.__dict__, "best_resnet18_audio.pt"}
```

test는 이 단계에서 보지 않습니다.

validation macro-F1 이 가장 높은 epoch의 weight만 best checkpoint로 남긴다.



학습 로그 해석

01	train_loss=2.31	val_acc=0.22	val_macro_f1=0.18
05	train_loss=1.02	val_acc=0.54	val_macro_f1=0.49
10	train_loss=0.20	val_acc=0.51	val_macro_f1=0.45

정상적인 변화

train loss가 줄고 validation 성능이 함께 올라간다.

과적합 신호

train loss는 계속 줄지만 validation macro-F1은 떨어진다. 가장 좋았던 checkpoint를 사용한다.



6 · 최종 test 평가

Py audio_classification_resnet18_dummy_dataset.ipynb

Dummy dataset notebook

6. 마지막 test 평가

accuracy, macro-F1, class별 결과와 confusion matrix를 확인합니다.

```
[ ] test = evaluate(loaders["test"])
    print({k: test[k] for k in ["loss", "accuracy", "macro_f1"]})
    print(classification_report(test["y"], test["pred"], target_names=LABELS, zero_division=0))

    cm = confusion_matrix(test["y"], test["pred"], normalize="true")
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, cmap="Blues", vmin=0, vmax=1, xticklabels=LABELS, yticklabels=LABELS)
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title("Normalized confusion matrix")
    plt.tight_layout()
    plt.show()
```

confusion matrix에서 자주 섞이는 class 쌍을 먼저 찾습니다.

test 결과를 보고 설정을 바꾸면 test가 사실상 validation이 된다.



7 · 오분류 파일 저장과 청취

Py audio_classification_resnet18_dummy_dataset.ipynb

Dummy dataset notebook

7. 오분류 파일 저장

틀린 clip을 표로 저장한 뒤 직접 듣고 원인을 annotation합니다.

```
[ ] errors = pd.DataFrame({"path": test["paths"], "true_idx": test["y"], "pred_idx": test["pred"]})
errors = errors[errors.true_idx != errors.pred_idx].copy()
errors["true_label"] = errors.true_idx.map(dict(enumerate(LABELS)))
errors["pred_label"] = errors.pred_idx.map(dict(enumerate(LABELS)))
errors["error_reason"] = ""
errors.to_csv("error_analysis.csv", index=False)
errors.head(20)
```

error_reason 옆에 noise, overlap, short, label, source, ambiguous 등을 기록합니다.

error_analysis.csv의 빈 error_reason을 채우는 것이 프로젝트의 핵심 분석이다.



Jupyter 문제 해결

- CUDA out of memory: batch size를 32에서 16 또는 8로 줄인다.
- kernel이 잘못 선택됨: 프로젝트 가상환경의 Python kernel로 바꾸고 재시작한다.
- audio decode error: WAV PCM, 16 kHz mono로 다시 변환한다.
- DataLoader worker error: 원인을 보기 위해 num_workers를 0으로 바꾸어 다시 실행한다.
- validation F1이 0에 가까움: label 오타, class 누락, split 표, 입력 spectrogram을 먼저 확인한다.
- 학습이 너무 느림: nvidia-smi와 notebook의 device: cuda 출력을 확인한다.



Notebook 실행 순서

1. Jupyter Lab에서 더미 데이터 notebook 열기
2. TASK 선택 및 metadata.csv 읽기
3. class 수와 group split 표 확인
4. spectrogram과 label 한 batch 확인
5. baseline 학습 및 best checkpoint 저장
6. validation으로 개선 방법 선택
7. 마지막에 test와 confusion matrix 계산
8. error_analysis.csv를 듣고 원인 기록

막히면 모델을 바꾸기 전에 폴더, metadata, split, 입력 그림 순서로 확인한다.



논문 · 데이터 · 공식 코드

1. Nagrani et al., "VoxCeleb: a large-scale speaker identification dataset," Interspeech 2017.
<https://www.robots.ox.ac.uk/~vgg/data/voxceleb/>
2. Desplanques et al., "ECAPA-TDNN," Interspeech 2020.
<https://arxiv.org/abs/2005.07143>
3. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," 2018.
<https://arxiv.org/abs/1804.03209>
4. Lekhak et al., "DogSpeak: A Canine Vocalization Classification Dataset," ACM MM 2025.
https://huggingface.co/datasets/ArlingtonCL2/Barkopedia_DOG_BREED_CLASSIFICATION_DATASET
5. Yin et al., "Towards Dog Bark Decoding," LREC-COLING 2024.
<https://aclanthology.org/2024.lrec-main.1432/>
6. He et al., "Deep Residual Learning for Image Recognition," CVPR 2016.
<https://arxiv.org/abs/1512.03385>



구현 참고 링크

- Torchvision ResNet-18 공식 문서
<https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html>
- Torchaudio MelSpectrogram 공식 문서
<https://docs.pytorch.org/audio/stable/generated/torchaudio.transforms.MelSpectrogram.html>
- PANNs 논문·공식 GitHub: 대규모 audio pattern recognition의 참고 baseline
https://github.com/qiuqiangkong/audioset_tagging_cnn
- VoxCeleb 공식 project page: in-the-wild speaker recognition 데이터 설계 참고
<https://www.robots.ox.ac.uk/~vgg/data/voxceleb/>

제공 파일

notebooks/audio_classification_resnet18_dummy_dataset.ipynb ·
templates/metadata_template.csv